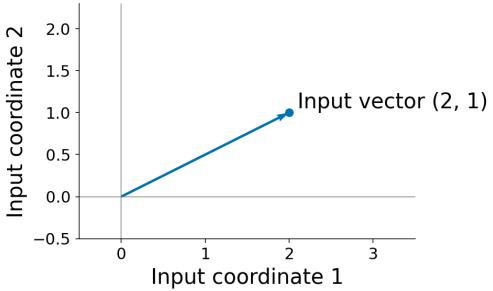
# From Gradients to Neural Networks

Connect familiar prediction rules to how more complex models learn.

1. Vectors $\rightarrow$ gradients $\rightarrow$ Gradient Boosting.
2. Support-vector geometry $\rightarrow$ neural-network layers $\rightarrow$ LSTM.
3. Conceptual extensions; existing project requirements stay the same.

# A vector is an ordered list

Scalar: one number. Vector: ordered numbers. Coordinate: one position. Dimension: number of positions.



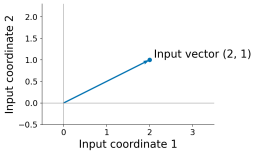

Coding Note, Example 6.1.

# A weighted sum connects the coordinates

A dot product multiplies matching coordinates and adds their products.

$$\mathbf{x} = (x_1, x_2), \qquad \mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2$$

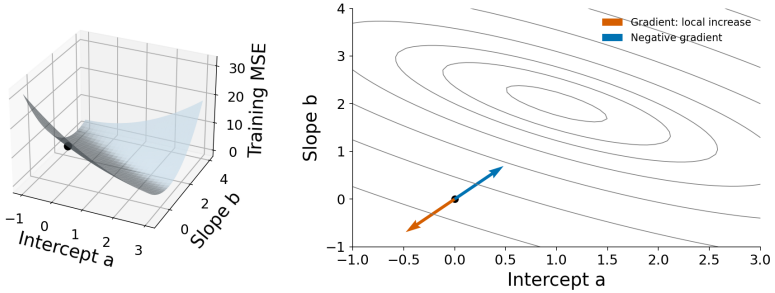
1. Weights (3,-1) pair with inputs (2,1).
2. The weighted sum is $3 \times 2 - 1 \times 1 = 5$.




Coding Note, Example 6.1.

# Draw a loss surface over model parameters

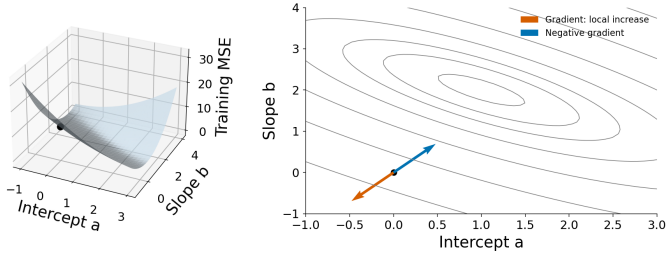
A parameter is adjusted during fitting. Loss summarizes prediction errors for a stated sample.




Coding Note, Example 6.2.

# Gradient: local loss changes in each direction

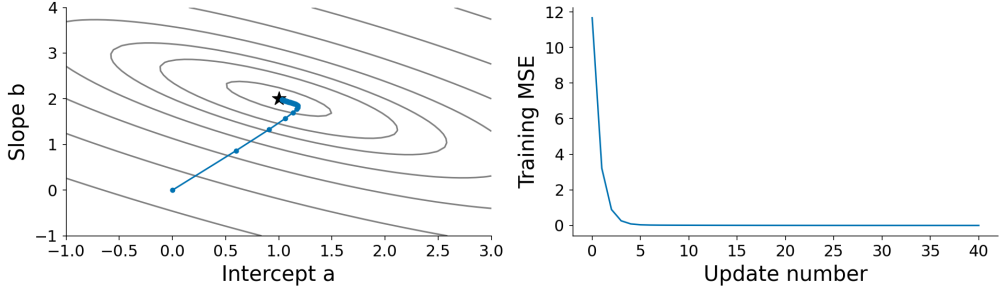
Change one setting a little; measure how the error responds. Collect those rates into a gradient.




Coding Note, Example 6.2.

# Gradient descent updates the parameters

Gradient descent repeatedly subtracts a scaled gradient from the current parameters.

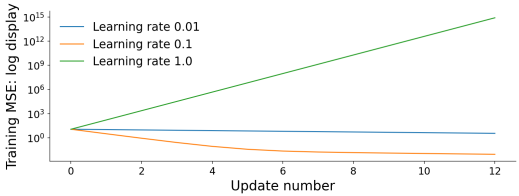



Coding Note, Example 6.3.

# The learning rate changes the path

Keep the problem and starting point fixed; vary only the update size.

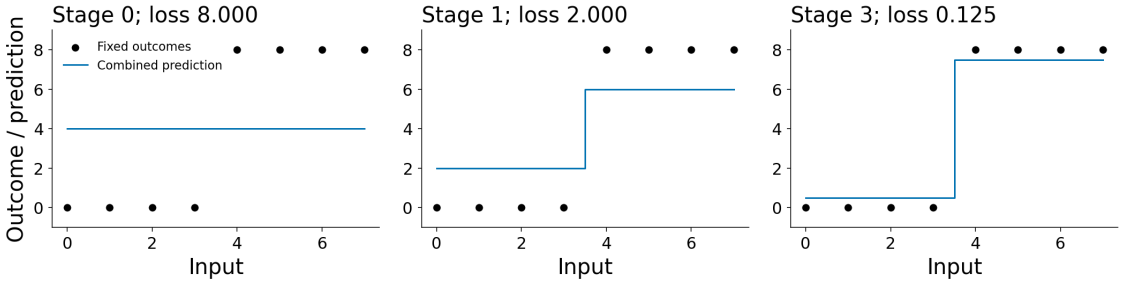
1. Small rate: slower progress.
2. Suitable rate: loss decreases in this example.
3. Large rate: overshooting can increase loss.




Coding Note, Example 6.4.

# Gradient Boosting adds a correction tree

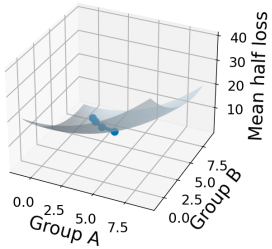
Gradient Boosting builds a combined rule by adding fitted corrections in stages.

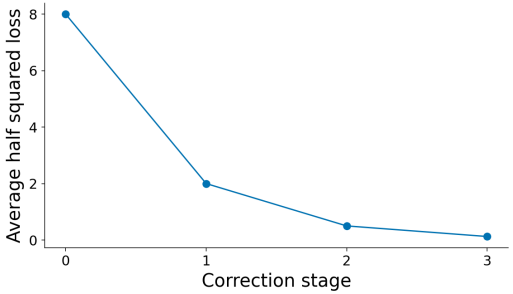



Coding Note, Example 6.5.

# Why does the correction point downhill?

For squared error, actual outcome minus prediction tells us which way to correct the prediction.



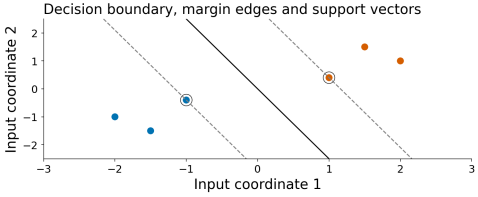




Coding Note, Example 6.6.

# SVM: learn a boundary between categories

Classification predicts a category. A linear SVM learns a score whose sign assigns a side of a boundary.

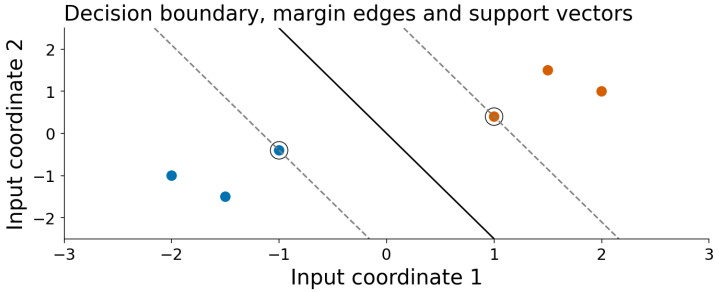
1. Input vector: two measured coordinates.
2. Weights and intercept define the score.
3. The zero-score line separates the two sides.




Coding Note, Example 6.7.

# Margin and support vectors

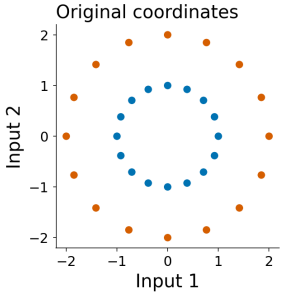
The margin is the strip around the dividing line. Supporting dots help determine its edges.

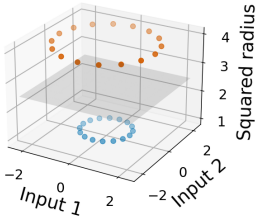



Coding Note, Example 6.7.

# A transformed representation can simplify separation

Calculate an extra input: distance from the center, squared. The outer ring rises higher.

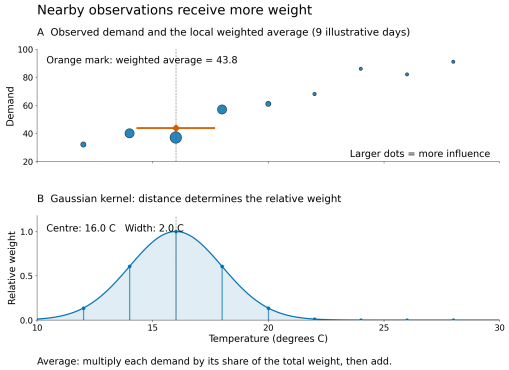



Coding Note, Example 6.8.

# Gaussian kernel: nearby observations matter more

Lower panel: distance weights. Upper panel: their influence on a local weighted average.

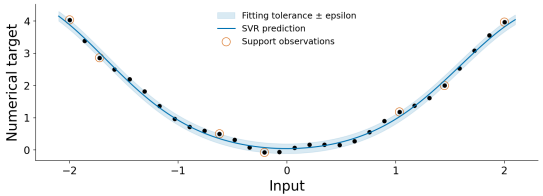



Coding Note, Example 6.8A.

# SVR predicts a numerical value

Support Vector Regression uses an epsilon-insensitive fitting loss for numerical targets.

1. Epsilon sets a tolerance in target units.
2. Only the part of absolute error beyond epsilon is penalized.
3. Evaluate forecasting uncertainty separately with saved outcomes.




Coding Note, Example 6.9.

# A neural-network unit has explicit arithmetic

A unit combines weighted inputs and a bias, then applies an activation.

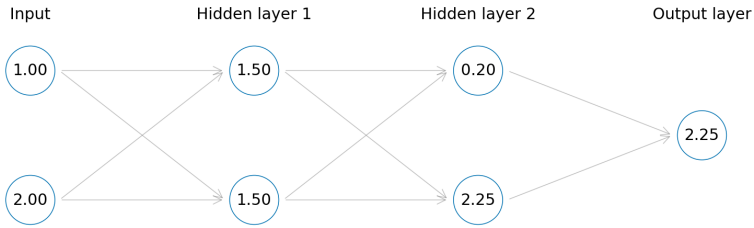
$$z = w_1 x_1 + w_2 x_2 + b, \qquad h = \max(0, z)$$




Coding Note, Example 6.10.

# Layers compose transformations

A layer groups units at one stage. Hidden layers lie between the inputs and the final output.

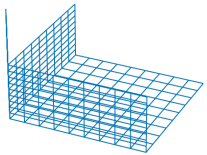



Coding Note, Example 6.11.

# Imagine deliberate folds in a marked sheet

Each fold changes the arrangement passed to the next fold, like successive numerical layers.

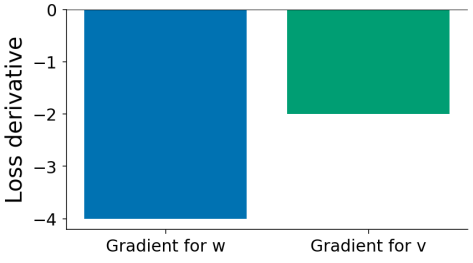
Folding analogy: compose deliberate transformations

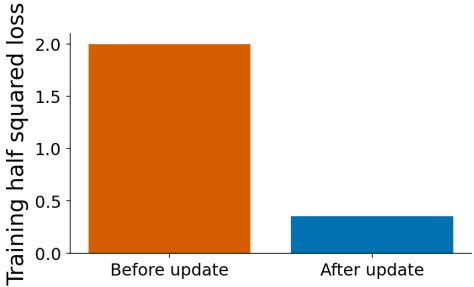


Coding Note, Example 6.11.

# Backpropagation traces how weights affect error

Trace how a weight changes the next number, the prediction and finally the error.

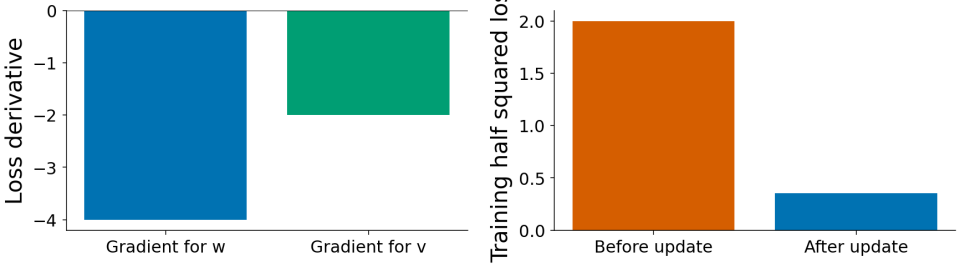



Coding Note, Example 6.12.

# Use the gradients to improve the prediction

Predict → measure error → work out adjustments → update → predict again.

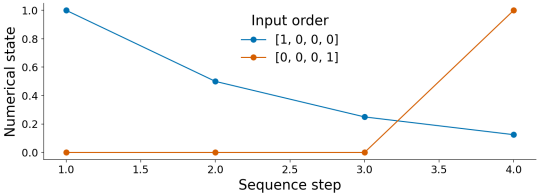



Coding Note, Example 6.12.

# An ordered sequence can carry information

A recurrent computation carries a numerical state from one step to the next.

1. Same numbers, different order: (1,0,0,0) and (0,0,0,1).
2. Same mean .25; final states .125 and 1 under the stated rule.
3. The state summarizes history according to its update calculation.

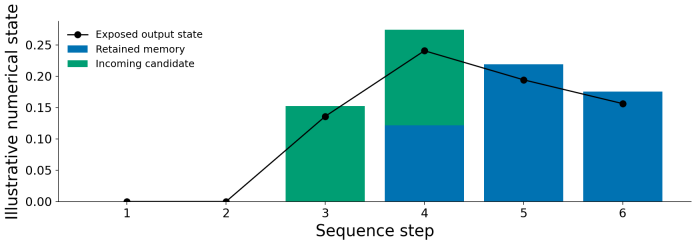



Coding Note, Example 6.13.

# LSTM: a neural network with a memory cell

A running numerical memory: keep some old information, add some new information, pass an output.

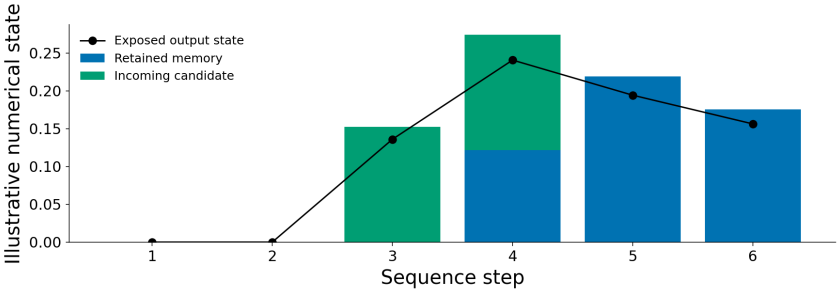
1. Forget gate: control retained old memory.
2. Input gate: control the incoming candidate contribution.
3. Output gate: control the exposed output state.




Coding Note, Example 6.14.

# Update the memory in meaningful blocks

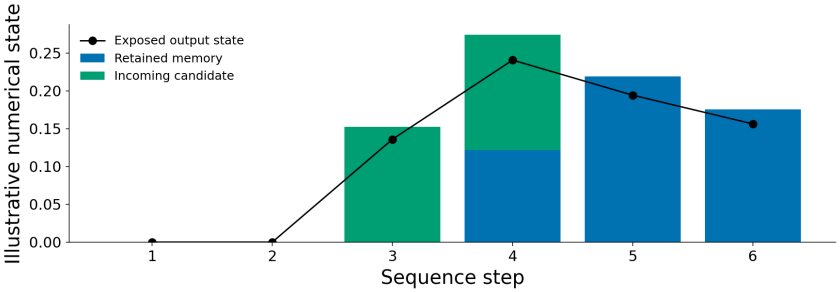
New memory = retained old memory + incoming contribution.




Coding Note, Example 6.14.

# Expose an output from the updated memory

Transform the updated memory, then multiply by the output gate.

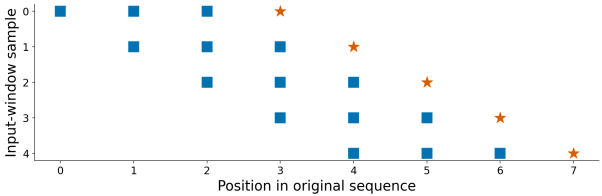



Coding Note, Example 6.14.

# Build sequence inputs for a later target

Common LSTM input dimensions are samples $\times$ time steps $\times$ features.

1. Eight observations; input length 3; five next-value targets.
2. Input shape (5,3,1): five windows, three steps, one feature.
3. Each target follows all observations in its input window.




Coding Note, Example 6.15.

# One bridge from linear models to sequence networks

Representation, loss and updates connect the methods introduced here.

1. Vectors organize inputs, parameters and gradients.
2. Boosting adds correction functions; SVM uses margins and representations.
3. Neural networks compose learned transformations; LSTM carries sequence memory.

# Continue your selected project

Choose one of the three cases released before Lecture 1.

1. Healthcare: disease progression. Finance/banking: profitability.
2. Business operations: next-day bike-sharing demand.
3. Follow the existing OLS, Ridge, Lasso and Random Forest tasks.

# Project submission and step credit

The final project is worth 200 points. Submit through the Blackboard link the instructor will open.

1. Deadline: Sunday, May 2, 2027, 11:59 PM Central Daylight Time.
2. Substantively attempt every required item: at least 100/200.
3. Remaining points reward correct steps, completeness and quality.